
Proyecto

Compilador para Código Binario

Fecha de asignación: 18 de Setiembre, 2026
Grupos: 3-4 estudiantes

Fecha de entrega: 14 de Octubre, 2026 (13:00)
Profesor: Jason Leitón Jiménez

1. Objetivo

El objetivo del proyecto es que el estudiante construya un compilador que traduce código fuente escrito en un lenguaje de programación diseñado por el grupo hacia código binario ejecutable. El estudiante debe diseñar una arquitectura de computadora personalizada, definir un conjunto de instrucciones propias, e implementar las fases necesarias para transformar el código fuente en ensamblador y posteriormente en código binario. Este proyecto enfatiza la traducción correcta de estructuras de control, la gestión de tablas de símbolos, el cálculo de saltos, y la resolución de referencias de memoria.

2. Requisitos del Proyecto

El compilador debe realizar las siguientes fases de traducción y utilizando **C como lenguaje de programación**:

2.1. Fase 1: Análisis Léxico

Se debe identificar y clasificar los tokens del código fuente. Esta fase debe reconocer:

- Palabras reservadas del lenguaje.
- Identificadores (nombres de variables, funciones, etiquetas).
- Constantes numéricas (enteros y posiblemente reales).
- Operadores y delimitadores.
- Comentarios (ignorarlos adecuadamente).
- Cualquier otra sentencia que sea necesaria y definida por el grupo.

2.2. Fase 2: Análisis Sintáctico

El compilador debe validar que el código fuente respete la gramática del lenguaje. Esta fase debe:

- Construir una representación interna del programa (árbol sintáctico abstracto o similar).
- Garantizar que las declaraciones, asignaciones, llamadas a funciones y estructuras de control sigan la sintaxis definida. (No es necesario que se haga validación de errores complejos porque es la primera versión)
- Garantizar que los bloques de código estén correctamente delimitados y anidados.

2.3. Fase 3: Análisis Semántico y Construcción de Tablas de Símbolos

El compilador desarrollado debe:

- Construir y mantener tablas de símbolos que registren variables, funciones, etiquetas y sus propiedades (tipo, dirección de memoria, ámbito).
- Resolver referencias: verificar que las variables se declaren antes de utilizarse y que las funciones se llamen con los argumentos correctos. (Esto no se revisará de manera exhaustiva, pero considero que puede ayudar en esta etapa).
- Asignar direcciones de memoria a las variables según el diseño de la arquitectura.
- Registrar puntos de salto (etiquetas) y sus direcciones en memoria para condicionales y ciclos.

2.4. Fase 4: Generación de Ensamblador

El compilador debe traducir el código fuente a ensamblador para la arquitectura definida. Esta fase debe:

- Generar instrucciones de ensamblador correspondientes a asignaciones, operaciones aritméticas y lógicas. Esto depende de cada grupo.
- Generar instrucciones de salto condicional e incondicional para estructuras de control (`if`, `while`, `for`). De acuerdo a las instrucciones de control definidas en la arquitectura.
- Generar código para llamadas a función, pasos de parámetros y retorno de valores.
- Mantener un contador de instrucciones para calcular direcciones de memoria.

2.5. Fase 5: Cálculo de Saltos y Resolución de Referencias

El proyecto debe:

- Identificar todas las etiquetas y sus direcciones numéricas correspondientes en el código binario.
- Resolver referencias a etiquetas: reemplazar nombres de etiquetas con direcciones de memoria reales.
- Calcular correctamente los desplazamientos relativos o direcciones absolutas para instrucciones de salto.
- Manejar casos como saltos dentro de condiciones anidadas y ciclos.

2.6. Fase 6: Generación de Código Binario

El compilador debe:

- Asignar códigos de operación (opcodes) a cada instrucción según la arquitectura diseñada.
- Codificar operandos (registros, direcciones de memoria, constantes).
- Generar el archivo binario final con la representación numérica del programa.

3. Especificación de la Arquitectura

Cada grupo debe diseñar su propia arquitectura y documentarla completamente:

3.1. Tamaño de Palabra

El compilador debe especificar el tamaño de la palabra de instrucción (p.ej., 16 bits, 32 bits) y explicar cómo se codifican las instrucciones dentro de esta palabra.

3.2. Conjunto de Registros

El compilador debe:

- Definir el número y propósito de registros (p.ej., acumulador, contador de programa, puntero de pila).
- Especificar qué registros se usan para almacenamiento temporal, parámetros y valores de retorno.
- Asignar un código numérico a cada registro.

3.3. Organización de Memoria

El compilador debe describir:

- El tamaño total de la memoria direccionable.
- Cómo se organiza la memoria (segmentos de código, datos, pila, heap).
- Cómo se asignan direcciones a variables y cómo se gestiona el espacio.

3.4. Conjunto de Instrucciones

El compilador debe definir un conjunto de al menos 15 instrucciones que incluya:

- Instrucciones de carga y almacenamiento (LOAD, STORE).
- Instrucciones aritméticas (ADD, SUB, MUL, DIV).
- Instrucciones lógicas (AND, OR, NOT).
- Instrucciones de comparación (CMP).
- Instrucciones de salto (JMP, JZ, JNZ, JG, JL).
- Instrucciones de control de flujo (CALL, RET).
- Instrucciones especiales (HALT, NOP).

Para cada instrucción, El compilador debe especificar:

- Su opcode (código numérico).
- Su formato: número de operandos y tipos permitidos.
- Ejemplo de uso en ensamblador.

Ejemplo de Tabla de Instrucciones

Instrucción	Opcode	Formato	Descripción
LOAD	01	LOAD Reg, Mem	Carga valor de memoria en registro
STORE	02	STORE Reg, Mem	Almacena registro en memoria
ADD	03	ADD Reg1, Reg2	Suma registros, resultado en Reg1
JZ	10	JZ Etiqueta	Salta si bandera cero
CALL	15	CALL Etiqueta	Llama subrutina
RET	16	RET	Retorna de subrutina
HALT	99	HALT	Detiene ejecución

4. Lenguaje de Entrada

El compilador debe definir un lenguaje de programación sencillo que incluya:

4.1. Elementos Básicos

- **Comentarios:** Especificar cómo se escriben (p.ej., `//`, `/* */`).
- **Declaración de variables:** Sintaxis y tipos de datos soportados.
- **Asignaciones:** Cómo se asignan valores a variables.
- **Expresiones:** Operaciones aritméticas, lógicas y de comparación.

4.2. Estructuras de Control

El lenguaje debe soportar:

- **Condicionales:** `if`, `else if`, `else`.
- **Ciclos:** Al menos `while` o `for`.
- **Funciones:** Declaración, parámetros, valor de retorno.
- **Llamadas a función:** Pasos de argumentos y recepción de resultados.

Ejemplo de Sintaxis

```
// Declarar variable
var x : entero = 10;

// Funci n simple
func suma(a : entero, b : entero) : entero {
    return a + b;
}

// Estructura de control
if (x > 5) {
    x = x + 1;
} else {
    x = x - 1;
}

// Ciclo
mientras (x < 20) {
    x = x + 2;
}
```

5. Invocación del Compilador

El compilador debe ser invocable desde la línea de comandos con la siguiente sintaxis general:

```
micomp [opciones] <archivo_fuente> [-o <salida>]
```

5.1. Opciones Principales

Opción	Descripción
-o <archivo>	Especifica el nombre del archivo binario de salida. Si se omite, se usa el nombre del fuente con extensión .bin .
-v	Modo detallado: imprime cada fase del compilador y decisiones importantes. S
-s	Genera archivo de ensamblador (.asm) además del binario.
-t	Imprime el árbol sintáctico generado (AST) en consola.
-m	Imprime la tabla de símbolos y asignaciones de memoria de manera legible y ordenada.
-x	Toma el código ensamblador en un archivo y genera el binario.
-p	Muestra los paquetes de instrucciones generados por el compilador en ensamblador y en binario.

5.2. Ejemplos de Uso

```
# Compilar y generar binario
micomp programa.ext -o programa.bin

# Generar binario y archivo ensamblador
micomp programa.ext -s

# Modo verbose con información detallada
micomp programa.ext -v -m

# Mostrar árbol sintáctico
micomp programa.ext -t
```

6. Salida del Compilador

6.1. Archivo Binario

El compilador debe generar un archivo binario que contenga:

- **Encabezado:** Información del programa (tamaño, punto de entrada, etc.).

- **Código:** Instrucciones en formato binario.
- **Datos:** Valores iniciales de variables globales.

6.2. Archivo de Ensamblador

El compilador puede generar un archivo intermedio en ensamblador que muestre:

- Instrucciones con mnemotécnicos legibles.
- Números de línea y direcciones.
- Tabla de símbolos (etiquetas y direcciones).

7. Ejercicios del Compilador

El compilador debe demostrar que su compilador funciona correctamente compilando los siguientes programas:

7.1. Ejercicio 1: Cálculo de Factorial

El compilador debe escribir un programa que calcule el factorial de un número utilizando:

- Variables locales.
- Condicionales (if).
- Ciclos (while o for).
- Operaciones aritméticas.

```
var n : entero = 5;
var resultado : entero = 1;
var i : entero = 1;

mientras (i <= n) {
    resultado = resultado * i;
    i = i + 1;
}
```

7.2. Ejercicio 2: Búsqueda en arreglo de 2 dimensiones (matriz)

El compilador debe escribir un programa que busque un elemento en un arreglo de dos dimensiones y retorne su posición:

- Declaración y uso de arreglos (de tamaño n).
- Ciclos con condición de salida.
- Operaciones de comparación.

7.3. Ejercicio 3: Programa Propuesto

El grupo propone un tercer programa que demuestre:

- Uso de funciones con parámetros y valores de retorno.
- Llamadas a múltiples funciones.
- Estructuras de control anidadas (condicionales dentro de ciclos).
- Operaciones complejas o algoritmo significativo.

8. Documentación del Compilador

El grupo debe entregar un documento que incluya:

8.1. Especificación de la Arquitecturav(mismo documento que Arquitectura de Computadores I)

- Tamaño de palabra e instrucción.
- Conjunto de registros con sus nombres y propósitos.
- Organización de memoria.
- Tabla completa de instrucciones con opcodes, formatos y ejemplos.

8.2. Especificación del Lenguaje de Entrada

- Definición de tokens (palabras reservadas, operadores, delimitadores).
- Sintaxis de elementos básicos (variables, asignaciones, expresiones).
- Sintaxis de estructuras de control y funciones.
- Ejemplo de comentarios y ámbito de variables.

8.3. Descripción de Algoritmos Clave

- Explicación del análisis léxico y cómo se reconocen tokens.
- Explicación del análisis sintáctico y construcción del AST.
- Explicación de la construcción de tablas de símbolos.
- Algoritmo de cálculo de saltos: cómo se resuelven las direcciones de etiquetas.
- Algoritmo de resolución de referencias: cómo se transforman nombres en direcciones.

8.4. Ejemplos de Ejecución

- Código fuente completo de los tres ejercicios.
- Ensamblador intermedio generado.
- Representación binaria (en hexadecimal legible).
- Traza de ejecución mostrando valores de variables y cambios de flujo.

8.5. Manejo de Casos Especiales

- Cómo se manejan saltos anidados (condicionales dentro de ciclos).
- Cómo se gestionan las llamadas a función y el retorno de valores.
- Cómo se resuelven conflictos en variables con el mismo nombre en diferentes ámbitos.
- Cómo se maneja la asignación de memoria para variables locales y globales.

9. Consideraciones Importantes

9.1. Restricciones de la Primera Versión

Para esta primera versión del compilador, El compilador **no** necesita implementar:

- Manejo de errores exhaustivo ni mensajes de error detallados.
- Optimización de código.
- Múltiples tipos de datos complejos.
- Gestión dinámica de memoria (pointers o referencias complejas).
- Módulos o archivos múltiples.
- Excepciones o tratamiento de errores en tiempo de ejecución.

9.2. Enfoque Principal

El compilador debe enfocarse correctamente en:

- Traducción correcta de instrucciones a código binario.
- Identificación precisa de cada token e instrucción del lenguaje.
- Construcción y manejo correcto de tablas de símbolos.
- Cálculo exacto de saltos y direcciones de memoria.
- Resolución correcta de todas las referencias a variables y etiquetas.

10. Entregable Final

El grupo debe entregar:

1. **Documento de especificación (PDF):** Incluyendo arquitectura, lenguaje, tabla de instrucciones, algoritmos clave y ejemplos.
2. **Código fuente del compilador:** Implementación de las fases del compilador en el lenguaje de programación elegido.
3. **Código fuente de los tres ejercicios:** Programas escritos en el lenguaje diseñado.

4. **Archivos generados:** Ensamblador y código binario de los tres ejercicios.
5. **Evidencia de ejecución:** Capturas de pantalla o salida mostrando la compilación exitosa y la generación correcta de binarios.